

Tài Liệu Mô Tả Mã Nguồn — Hệ Thống K8sSecDaP

Phiên bản: 2.0 · Ngày tổng hợp: 2026-06-12

Mục Lục

- 1. Tổng Quan Hệ Thống
- 2. Cấu Trúc Repo Umbrella và Các Submodule
- 3. Kiến Trúc Tổng Thể
- 4. Mô Tả Mã Nguồn — Đào Sâu Từng Submodule
 - 4.1 K8sSecDaP-collector (eBPF)
 - 4.2 K8sSecDaP-pipeline (C++ DSA)
 - 4.3 K8sSecDaP-soc (Go / Python)
- 5. Luồng Thực Thi End-to-End
 - 5.1 Luồng Phát Hiện (Detect)
 - 5.2 Luồng Ngăn Chặn (Prevent)
- 6. Config Platform
- 7. Cơ Sở Dữ Liệu và Lưu Trữ
- 8. Observability
- 9. Dependencies và Công Cụ
- 10. Hướng Dẫn Build và Chạy
- 11. Kết Quả Đánh Giá Đã Đo Được
- 12. Các Vấn Đề Đã Biết và Hạn Chế
- 13. Glossary — Thuật Ngữ Viết Tắt

1. Tổng Quan Hệ Thống

1.1 K8sSecDaP là gì?

K8sSecDaP — *Kubernetes Security Data Pipeline* — là một hệ thống phát hiện và ngăn chặn hành vi **quét cổng (port-scan)** và **dịch chuyển ngang (lateral movement)** trong cụm Kubernetes, được xây dựng như đồ án tốt nghiệp cấp đại học (ĐATN).

"Phát hiện và ngăn chặn hành vi quét cổng/dịch chuyển ngang trong Kubernetes bằng eBPF và cấu trúc dữ liệu luồng"

Nguồn: K8sSecDaP/plans/plan-thesis.md:56

1.2 Bài Toán Giải Quyết

Trong một cụm Kubernetes sử dụng mạng flat (mặc định không có NetworkPolicy), một Pod đã bị chiếm quyền kiểm soát có thể:

- 1. Quét các Pod khác để tìm cổng mở (port-scan).
- 2. Dịch chuyển ngang sang các Pod/Service khác (lateral movement).

K8sSecDaP phát hiện hành vi này ở tầng kernel (không cần sửa ứng dụng) và tự động cách ly nguồn tấn công bằng NetworkPolicy của Kubernetes.

1.3 Threat Model (Mô Hình Đe Dọa)

Yếu tố	Mô tả
Tài sản được bảo vệ	Các Pod/Service trong namespace <code>zt-targets</code>
Kẻ tấn công	Một Pod đã bị chiếm trong cluster, thực hiện port-scan hoặc lateral movement
Khả năng quan sát	Mọi TCP connect đi qua kernel của node → eBPF bắt được tại <code>tcp_v4_connect</code>
Giả định	CNI có hỗ trợ NetworkPolicy (Calico); kernel ≥ 5.8 với BTF; IPv4; attacker không có quyền root trên node host
Ngoài mô hình	Tấn công L7, kernel exploit, attacker đã thoát container

Nguồn: K8sSecDaP/plans/plan-thesis.md:102-107

2. Cấu Trúc Repo Umbrella và Các Submodule

2.1 Repo Umbrella

Đường dẫn: `/home/john/dev/_split/K8sSecDaP`

Đây là repo chính (monorepo-style) chứa toàn bộ hệ thống dưới dạng **Git submodule**. File cấu hình submodule:

K8sSecDaP/.gitmodules

```
[submodule "infra"]      → K8sSecDaP-infra      (Ansible, provisioning cluster)
[submodule "deploy"]     → K8sSecDaP-deploy     (GitOps, ArgoCD apps)
[submodule "soc"]        → K8sSecDaP-soc        (Go services, manifests)
[submodule "pipeline"]   → K8sSecDaP-pipeline   (C++ DSA, engine phân tích)
[submodule "collector"]  → K8sSecDaP-collector  (eBPF kernel module)
[submodule "report"]     → K8sSecDaP-report     (LaTeX báo cáo)
```

2.2 Bảng Các Submodule

Submodule	Đường dẫn (local)	Ngôn ngữ	Vai trò
K8sSecDaP-collector	collector/	C, C++ (eBPF/libbpf)	Thu thập sự kiện TCP từ kernel qua kprobe, phát ra JSON
K8sSecDaP-pipeline	pipeline/	C++17, CMake	Phân tích sự kiện: CMS, Tarjan SCC, BFS blast-radius, LPM
K8sSecDaP-soc	soc/	Go, Python (Flask)	Aggregator, incident-service, web-console, manifests K8s
K8sSecDaP-infra	infra/	Ansible (YAML)	Provisioning cluster K8s bằng kubeadm + Calico + Tailscale
K8sSecDaP-deploy	deploy/	YAML (Helm/Kustomize/ArgoCD)	GitOps — quản lý toàn bộ platform trên cluster
K8sSecDaP-report	report/	LaTeX	Báo cáo học thuật (BC1 + BC2/thesis)

2.3 Nội Dung Umbrella Repo (Ngoài Submodule)

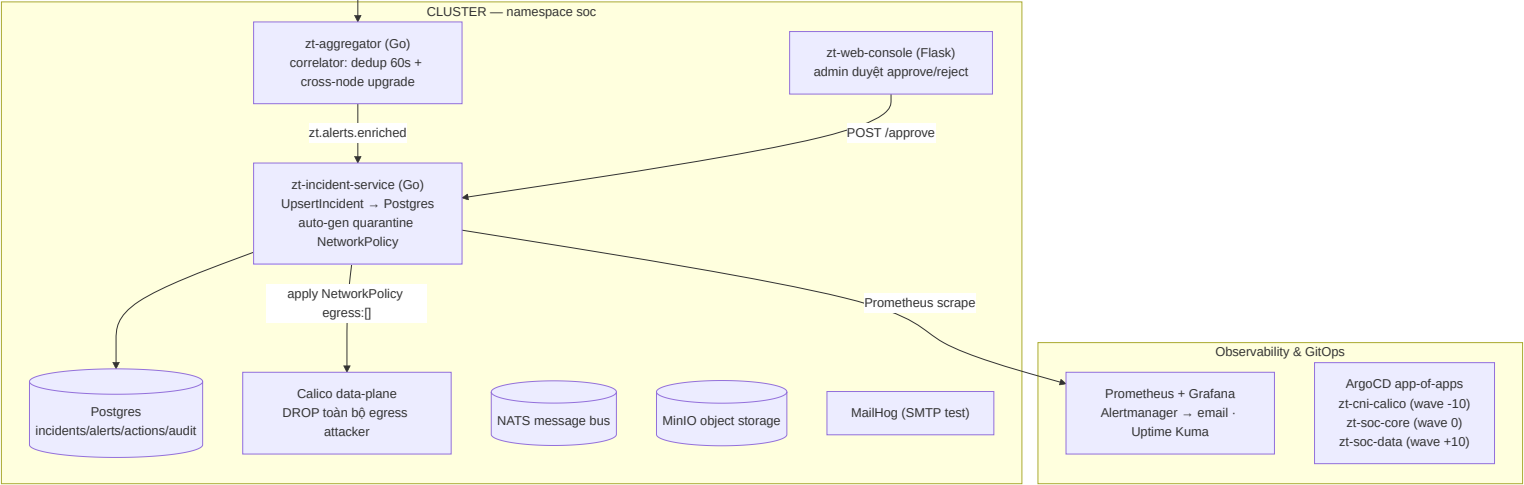
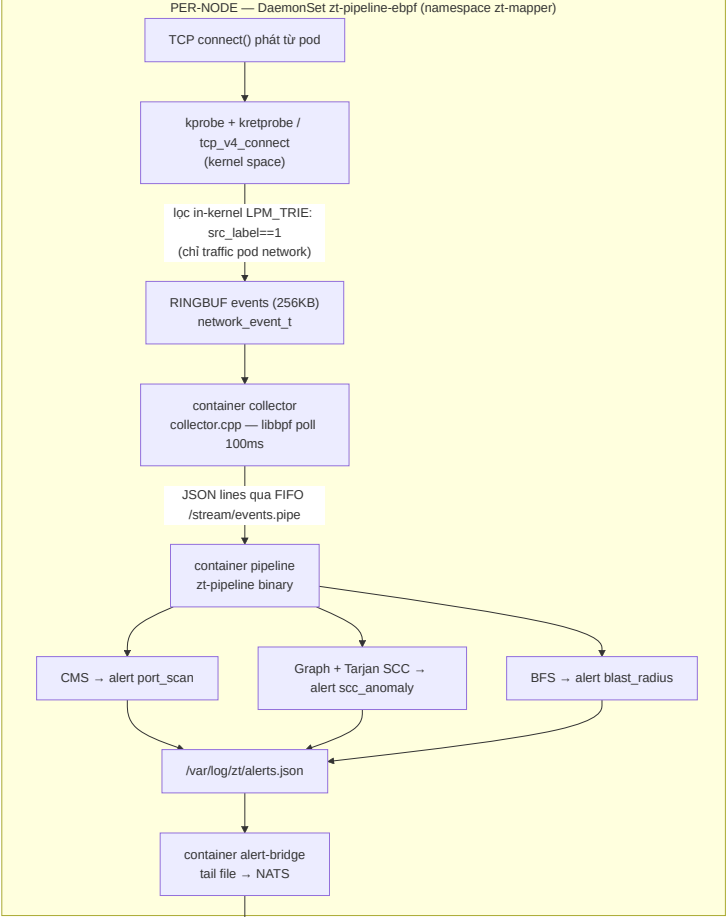
Thư mục/File	Nội dung
plans/plan-thesis.md	Master plan đề án — checklist tiến độ, kiến trúc, milestones
plans/plan.md , plan-2.md	Kế hoạch giai đoạn 1–5 DSA (tham chiếu gốc)

Thư mục/File	Nội dung
docs/phase1-6/	Tài liệu lý thuyết: CMS, SCC, BFS, LPM, eBPF, Zero-Trust
config/pipeline.yaml	Config pipeline local (dùng cho dev/test)
config/portscan.yaml	Config demo port-scan
scripts/	Python/Bash: generate synthetic events, tcpdump parser, plot benchmarks, attack scenarios
tools/portscan-pipeline/	Standalone C++ tool demo port-scan (tách khỏi pipeline chính)
visualization/	Python renderer đồ thị kết nối

3. Kiến Trúc Tổng Thể

3.1 Sơ Đồ Kiến Trúc

Hệ thống chia làm ba tầng: tầng **per-node** (DaemonSet thu thập + phân tích ngay tại kernel của từng node), tầng **cluster** (SOC tập trung tương quan và ra quyết định), và tầng **observability/GitOps** (giám sát + triển khai).



3.2 Namespace Kubernetes

Namespace	Nội dung
zt-mapper	DaemonSet zt-pipeline-ebpf (collector + pipeline + alert-bridge)
soc	NATS, Postgres, aggregator, incident-service, web-console, MinIO, MailHog
zt-targets	Pod nạn nhân (web, api) + Pod attacker (demo)
monitoring	kube-prometheus-stack (Prometheus, Grafana, Alertmanager)
argocd	ArgoCD controller

Namespace	Nội dung
kyverno	Kyverno admission controller + ClusterPolicies
tigera-operator	Tigera Operator quản lý Calico CNI
traefik	Traefik Ingress controller
cert-manager	cert-manager
sealed-secrets	Bitnami Sealed Secrets

4. Mô Tả Mã Nguồn — Đào Sâu Từng Submodule

Phần này không liệt kê file overview, mà giải thích cặn kẽ mã nguồn thật: mỗi cơ chế core nói rõ **nó làm gì, tại sao phải làm thế, và hoạt động ra sao ở mức dòng code**.

4.1 K8sSecDaP-collector — eBPF

Đường dẫn: K8sSecDaP/collector/ · **Ngôn ngữ:** C (eBPF kernel), C++ (user-space, libbpf) · **Build:** Docker multi-stage + build.sh + Makefile

```
collector/
├─ src/kern/tcp_connect.bpf.c    # Chương trình eBPF (kernel space)
├─ src/user/collector.cpp        # User-space loader (libbpf)
├─ Dockerfile                   # Multi-stage: builder + runtime
├─ build.sh                     # Stage BTf → docker build
└─ Makefile                     # Native build path (dev)
```

4.1.1 Cặp kprobe/kretprobe của tcp_v4_connect và map sock_store

Module hook vào hàm kernel `tcp_v4_connect` bằng **hai** điểm bẫy riêng biệt (`tcp_connect.bpf.c:64-144`). Lý do phải dùng hai điểm thay vì một: tại thời điểm hàm mới được gọi (entry), kernel **chưa** điền giá trị vào các trường địa chỉ của `struct sock` . Cụ thể, `tcp_v4_connect` nhận một `struct sock *sk` chứa socket chưa kết nối, rồi mới ghi địa chỉ đích (`skc_daddr`), cổng đích (`skc_dport`) và địa chỉ nguồn (`skc_rcv_saddr`) trong thân hàm. Nếu đọc ngay ở entry, giá trị sẽ là rác hoặc zero.

Kprobe entry (`tcp_connect.bpf.c:64-70`) chỉ làm một việc: lưu con trỏ `sk` vào map `sock_store` theo key là `pid_tgid` :

```
SEC("kprobe/tcp_v4_connect")
int BPF_KPROBE(tcp_v4_connect_entry, struct sock *sk)
{
    __u64 pid_tgid = bpf_get_current_pid_tgid();
    bpf_map_update_elem(&sock_store, &pid_tgid, &sk, BPF_ANY);
    return 0;
}
```

Key `pid_tgid` là giá trị 64-bit: 32 bit cao là TGID (process ID), 32 bit thấp là thread ID. Dùng cả 64 bit đảm bảo phân biệt được nhiều thread trong cùng một process đồng thời gọi `connect()`.

Kretprobe (`tcp_connect.bpf.c:75-144`) chạy khi `tcp_v4_connect` chuẩn bị return — lúc này socket đã có đầy đủ thông tin. Nó dùng cùng `pid_tgid` để tra lại con trỏ `sk` và **xóa** entry ngay sau khi dùng để tránh rò rỉ:

```
struct sock **skp = bpf_map_lookup_elem(&sock_store, &pid_tgid);
if (!skp)
    return 0;
struct sock *sk = *skp;
bpf_map_delete_elem(&sock_store, &pid_tgid);
```

Map `sock_store` (`tcp_connect.bpf.c:53-58`) là `BPF_MAP_TYPE_HASH` với `max_entries = 4096`, đủ chứa tối đa 4096 lần `connect()` đang chờ kết quả song song. Đây chính là "keo dính" hai điểm bấy lại với nhau.

4.1.2 BPF CO-RE và byte-order khi đọc trường từ `struct sock`

Tại `tcp_connect.bpf.c:97-102`, code đọc ba trường bằng macro `BPF_CORE_READ_INTO`:

```
BPF_CORE_READ_INTO(&dst_ip, sk, __sk_common.skc_daddr);
BPF_CORE_READ_INTO(&dst_port, sk, __sk_common.skc_dport);
BPF_CORE_READ_INTO(&src_ip, sk, __sk_common.skc_rcv_saddr);
dst_port = __builtin_bswap16(dst_port);
```

CO-RE (Compile Once – Run Everywhere) giải quyết vấn đề: mỗi phiên bản kernel có thể đổi layout struct nội bộ (offset, padding), trong khi eBPF bytecode compile ra lại chứa offset cứng. CO-RE tách hai pha: (1) compile-time dùng `VMLINUX_H` (sinh từ BTF của máy build) để tạo *relocation metadata* trong `.bpf.o`; (2) load-time, libbpf đọc BTF của kernel **đang chạy** từ `/sys/kernel/btf/vmlinux`, so offset thực tế và **patch lại** bytecode trước khi nạp. Nhờ vậy cùng một `tcp_connect.bpf.o` chạy đúng trên nhiều kernel mà không cần recompile. Macro `BPF_CORE_READ_INTO` sinh ra `bpf_probe_read_kernel()` kèm relocation record — đây là cách truy cập an toàn được verifier chấp nhận, thay vì dereference raw pointer trực tiếp.

Vấn đề byte-order (điểm dễ nhầm): trong kernel, IP và port trong socket đều ở *network byte order* (big-endian) vì TCP/IP stack nhận thẳng từ network frame. Code giữ nguyên `src_ip / dst_ip` ở network order (ghi chú dòng 133: *"IPs stored in network byte order for consistency"*), và phía user-space `ip_to_str` (`collector.cpp:40-44`) dùng `inet_ntop(AF_INET, ...)` hiểu đúng đầu vào network order. Riêng `dst_port` đã được `__builtin_bswap16` về *host order* ngay trong kernel (dòng 102) trước khi điền vào struct. Đây là sự **không đồng nhất convention nội bộ** của struct cần ghi nhớ: hai IP là network order, nhưng port là host order khi ra tới JSON (`"dst_port":8080` đọc được).

4.1.3 Xử lý giá trị return — ý nghĩa của `-EINPROGRESS`

Kretprobe lọc sớm tại `tcp_connect.bpf.c:79`:

```
if (ret != 0 && ret != -115)
    return 0;
```

`-115` là `-EINPROGRESS`. Cần hiểu hai chế độ socket: với **blocking socket**, `tcp_v4_connect` đợi handshake và return `0` (thành công) hoặc lỗi âm. Với **non-blocking socket** (rất phổ biến trong K8s: gRPC, HTTP/2, các thư viện async), nó gửi SYN rồi return ngay `-EINPROGRESS` — đây **không phải lỗi** mà nghĩa là kết nối đã được *khởi xướng thành công* và đang chạy bất đồng bộ. Nếu bỏ qua `-EINPROGRESS`, module sẽ mất phần lớn event thực tế, nên chấp nhận cả `0` lẫn `-115` là điều kiện cần để bắt đủ traffic.

4.1.4 Bộ lọc in-kernel bằng `cidr_map` (LPM_TRIE)

Map `cidr_map` (`tcp_connect.bpf.c:36-42`) là `BPF_MAP_TYPE_LPM_TRIE` với flag `BPF_F_NO_PREALLOC` (bắt buộc với `LPM_TRIE` vì node được cấp phát động, không prealloc như hash map). Key là `struct lpm_key` (`tcp_connect.bpf.c:27-30`) gồm `prefixlen` (số bit có nghĩa) + `addr` (IPv4 network order) — đúng format kernel yêu cầu.

Lookup với `prefixlen=32` (host address) sẽ trả entry có prefix **dài nhất** khớp. Cơ chế lọc nguồn (`tcp_connect.bpf.c:109-115`):

```
struct lpm_key src_key = { .prefixlen = 32, .addr = src_ip };
__u32 *src_label = bpf_map_lookup_elem(&cidr_map, &src_key);
if (!src_label || *src_label != 1)
    return 0;
```

Chỉ khi `src_ip` thuộc map **và** label là `1` (pod network) thì event mới đi tiếp. Traffic từ host network của node (label 3), loopback (label 5), hoặc IP không thuộc CIDR nào (`NULL`) đều bị drop **ngay trong kernel**.

Tại sao lọc trong kernel thay vì user-space: nếu emit mọi TCP connect toàn node, volume sẽ rất lớn (kubelet, kube-proxy, containerd, systemd, và chính collector). Lọc sớm giảm số `bpf_ringbuf_reserve` thành công, giảm dữ liệu copy kernel → user, và đặc biệt giảm context switch + CPU wake-up của collector. Đây là nguyên tắc thiết kế eBPF: đẩy policy xuống càng gần nguồn dữ liệu càng tốt. Lưu ý: lookup cho địa

chỉ **đích** (`tcp_connect.bpf.c:119-126`) có thực hiện nhưng kết quả không dùng để filter — comment ghi *"For now, we emit ALL events and let user space classify"*, để ngỏ cho pipeline downstream phân loại.

4.1.5 Ring buffer — cơ chế reserve/submit

Từ `tcp_connect.bpf.c:128-141` :

```
struct network_event_t *evt;
evt = bpf_ringbuf_reserve(&events, sizeof(*evt), 0);
if (!evt)
    return 0;
evt->src_ip = src_ip;
/* ... fill fields ... */
bpf_ringbuf_submit(evt, 0);
```

`BPF_MAP_TYPE_RINGBUF` (kernel ≥ 5.8) hoạt động theo mô hình *producer reserves then writes directly*:

`bpf_ringbuf_reserve` cấp phát vùng nhớ ngay trong ring buffer và trả con trỏ thẳng vào đó (ghi không cần copy thêm), `bpf_ringbuf_submit` đánh dấu sẵn sàng cho user-space đọc. So với perf buffer cũ, ring buffer có hai ưu điểm: (1) là một vùng nhớ chia sẻ liên tục được memory-map vào cả kernel lẫn user, không cần `copy_to_user` ; (2) chỉ một buffer dùng chung (perf buffer có buffer riêng mỗi CPU → dễ lệch tải). Kích thước 256 KB = 64 page (phải là bội số page 4096B). Nếu buffer đầy, `reserve` trả NULL và event bị drop — đó là lý do có `if (!evt) return 0` .

4.1.6 User-space collector.cpp — trình tự và điểm để nhaim

Trình tự **open** → **load** → **attach** (`collector.cpp:118-150`):

- `bpf_object__open_file("tcp_connect.bpf.o", ...)` (dòng 118): đọc ELF, phân tích sections/maps/programs nhưng **chưa** nạp gì vào kernel. Đường dẫn tương đối nên collector phải chạy từ thư mục chứa `.bpf.o` (Dockerfile đặt `WORKDIR /app`).
- `bpf_object__load(obj)` (dòng 127): bước nạp thật — libbpf chạy CO-RE relocation, chạy verifier, cấp phát map, nạp bytecode.
- `bpf_program__attach` cho từng program (dòng 134-135): tự nhận biết prefix `kprobe/` hay `kretprobe/` từ tên section ELF để attach đúng chỗ.

`populate_cidr_map` (`collector.cpp:70-97`) hardcoded các CIDR:

```
{ {16, htonl(0x0AF40000)}, 1, "pod_network" }, // 10.244.0.0/16
{ {12, htonl(0x0A600000)}, 2, "service_network" }, // 10.96.0.0/12
{ {16, htonl(0xC0A80000)}, 3, "node_network" }, // 192.168.0.0/16
{ {12, htonl(0xAC100000)}, 4, "private_172" }, // 172.16.0.0/12
{ {8,  htonl(0x7F000000)}, 5, "loopback" }, // 127.0.0.0/8
```

Điểm dễ nhầm: `htonl()` cần thiết vì literal trong code là host-order, phải đổi sang network order để khớp cách kernel lưu địa chỉ. Còn khi kernel lookup với `src_key.addr = src_ip`, `src_ip` đã là network order (đọc thẳng từ `sk_rcv_saddr`), nên không `bswap` nữa — hai phía cùng network order, lookup khớp.

Vòng poll (`collector.cpp:181-190`) dùng `ring_buffer__poll(rb, 100)` (timeout 100ms). Khi nhận `SIGINT/SIGTERM`, `sig_handler` set `exiting = true`; nếu poll đang block, signal làm hàm trả `-EINTR` và code `break` ngay. Output JSON in ra **stdout** (dòng 60-63), còn diagnostic in ra **stderr** (dòng 177-178), để pipeline ngoài (`kubectl logs`, log aggregator) chỉ đọc JSON thuần từ stdout.

4.1.7 Build pipeline — vmlinux.h, BTF, và lý do không commit

`vmlinux.h` chứa toàn bộ type definitions của kernel, sinh từ BTF qua `bpftool btf dump file /work/btf/vmlinux format c` (Dockerfile:35). Vì nó phụ thuộc phiên bản kernel, đây là artifact build-time, **không commit** vào git.

`build.sh` phải **stage BTF trước docker build** vì Docker daemon không truy cập được `/sys/` của host (virtual filesystem của kernel). Script copy BTF vào build context:

```
install -m 0644 "$BTF_SRC" btf/vmlinux
docker build --build-arg ARCH="$ARCH" -t "$IMAGE" .
```

`BTF_SRC` mặc định `/sys/kernel/btf/vmlinux` — điều kiện tiên quyết: kernel máy build phải có `CONFIG_DEBUG_INFO_BTF=y`. Tính portable CO-RE (Dockerfile comment dòng 12-14): `vmlinux.h` chỉ cấp *type definitions* lúc compile, còn *field offset thực tế* được relocate lúc load theo BTF của kernel **target** — nên image build trên kernel 6.1 vẫn chạy đúng trên node kernel 5.15. Runtime image (Dockerfile:51-63) chỉ giữ `collector + .bpf.o + ba shared lib (libbpf1, libelf1, zlib1g)`, không có compiler/bpftool → giảm attack surface. Makefile cung cấp đường build native (không Docker) cho dev; khi `make run` cần `sudo` (`CAP_BPF + CAP_PERFMON`, hoặc `CAP_SYS_ADMIN` trên kernel cũ).

4.1.8 struct network_event_t và ba BPF maps

```
struct network_event_t {           // tcp_connect.bpf.c:17-23
    __u32 src_ip;                   // network byte order
    __u32 dst_ip;                   // network byte order
    __u16 dst_port;                 // HOST byte order (sau bswap16)
    __u32 pid;                      // TGID
    __u64 timestamp_ns;             // ns từ boot (CLOCK_MONOTONIC)
};
```

Struct này phải khớp tuyệt đối với định nghĩa phía user-space (`collector.cpp:24-30`). Ba map ba mục đích khác nhau: `cidr_map` (LPM_TRIE) — bảng tra CIDR để phân loại/lọc; `events` (RINGBUF) — kênh stream một chiều kernel → user; `sock_store` (HASH) — bộ nhớ tạm per-thread tồn tại giữa entry và return.

4.2 K8sSecDaP-pipeline — C++ DSA

Đường dẫn: K8sSecDaP/pipeline/ · **Ngôn ngữ:** C++17, CMake · **Test:** Google Test + Google Benchmark

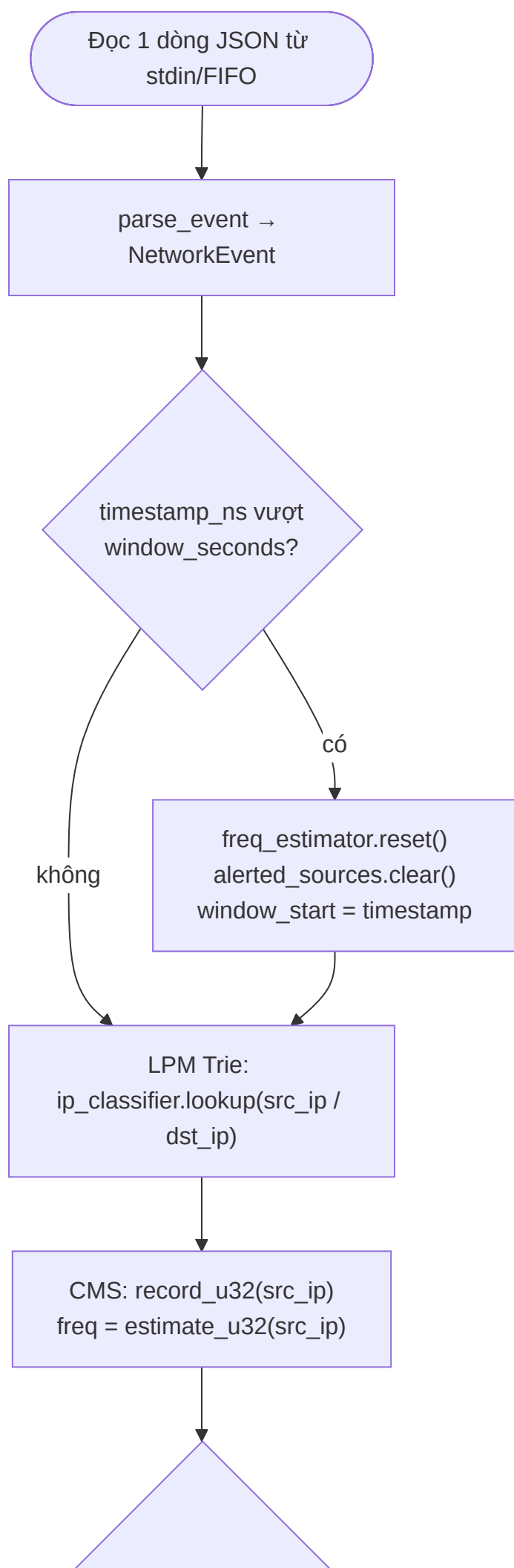
```
pipeline/
├─ pipeline/src/engine.cpp          # MAIN: event loop + orchestration
├─ pipeline/src/config_loader.cpp   # Parser YAML thủ công
└─ libdsa/
    ├─ include/dsa/frequency/      estimator.h, count_min_sketch.h, hash_map_exact.h, heavy_hitti
    ├─ include/dsa/graph/          directed_graph.h, tarjan.h, kosaraju.h, bfs.h, dijkstra.h, sc
    ├─ include/dsa/trie/            lpm_trie.h, ip_classifier.h
    ├─ include/dsa/registry.h       # Strategy + Singleton template
    └─ src/                         # Implementations + tests/ + benchmarks/
```

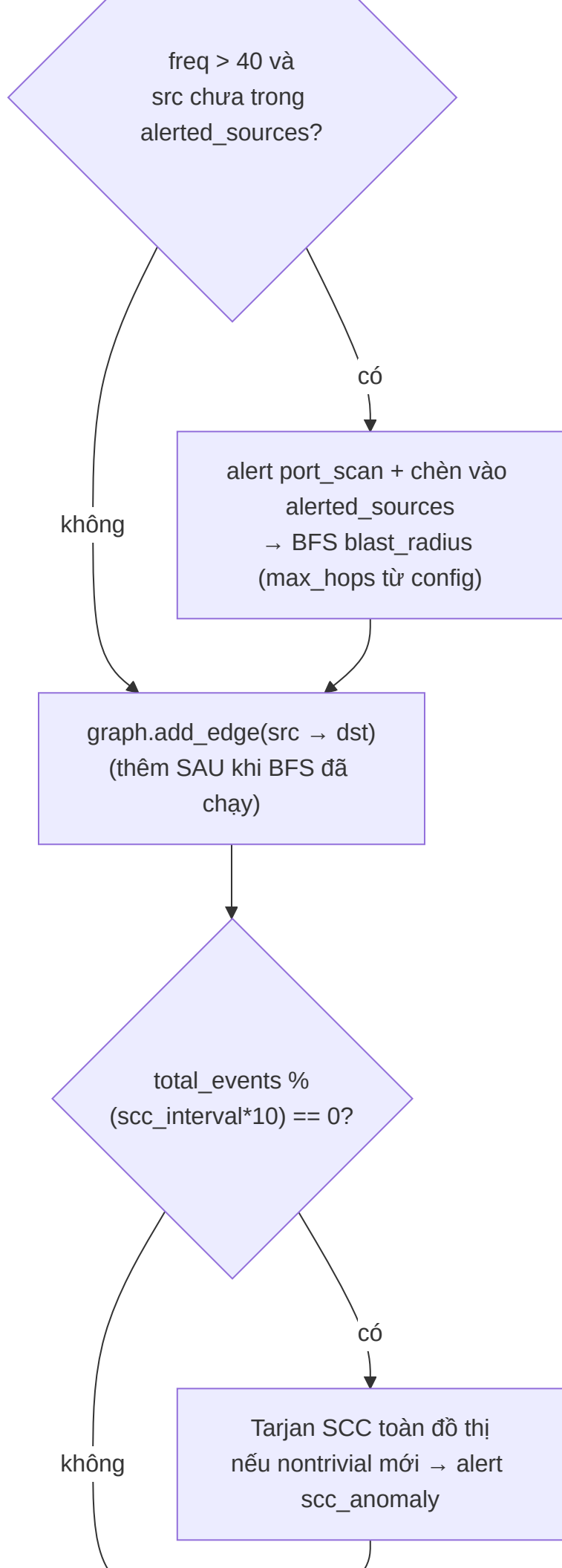
4.2.1 Khởi tạo và vòng event loop (engine.cpp)

`main()` (`engine.cpp:221`) khởi tạo theo thứ tự nghiêm ngặt. Dòng đầu tiên (dòng 237) là `dsa::register_all_algorithms()` — **bắt buộc** trước mọi `Registry::create()` , nếu không `Registry` trống và ném `std::runtime_error` . Tiếp theo là `register_all_alert_handlers()` , rồi nạp config YAML, và tạo **bốn** đối tượng thuật toán qua bốn `Registry::instance().create()` (dòng 287-304): `freq_estimator` , `scc_detector` , `reachability` , `ip_classifier` . Cả bốn là `std::unique_ptr` tới interface trừu tượng — engine **không biết** cụ thể đang dùng `CountMinSketch` hay `HashMapExact` , chỉ biết interface `Estimator` . Sau đó LPM Trie được điền CIDR từ config (dòng 307-311), và `AlertHandlerChain` được dựng (dòng 339-345: nếu danh sách trống thì tự tạo file handler trở `cfg.alerts_file`).

Vòng lặp chính (dòng 387-533) đọc stdin/file từng dòng bằng `std::getline()` , đưa vào `parse_event()` (dòng 87-105) — một bộ parse JSON tối giản không dùng thư viện ngoài (`json_string_value()` dòng 45-67 tìm "key" , nhảy qua `:` , đọc chuỗi nếu gặp `"` hoặc đọc số tới `,`). IP chuyển sang `uint32_t` big-endian bằng `parse_ip()` (dòng 69-78).

Sơ đồ xử lý mỗi event (bốn thuật toán phối hợp):







Event tiếp theo

Tumbling window (cửa sổ lăn). Đây là cơ chế quan trọng nhất để tránh trạng thái tích lũy vô hạn (engine.cpp:397-403):

```
uint64_t window_ns = (uint64_t)cfg.window_seconds * 1'000'000'000ULL;
if (evt.timestamp_ns - window_start_ns > window_ns) {
    freq_estimator->reset();
    alerted_sources.clear();
    window_start_ns = evt.timestamp_ns;
}
```

Khi cửa sổ hết hạn, **đồng thời** reset bảng đếm tần số (CMS về 0) và xóa alerted_sources . Đây là *tumbling window* (không chồng lấn): window mới bắt đầu ngay tại event vượt ngưỡng thời gian, không phải tại biên cố định. Quan trọng: **đồ thị DirectedGraph và prev_nontrivial_sccs KHÔNG bị reset** — chúng tích lũy xuyên suốt phiên chạy vì phản ánh topology mạng lâu dài, không phải tần số ngắn hạn.

Chống alert trùng (alerted_sources). Là std::unordered_set<uint32_t> các IP nguồn đã alert trong window hiện tại (engine.cpp:415-417). Mỗi IP chỉ tạo tối đa một port_scan (và một blast_radius) mỗi cửa sổ, dù tần số tiếp tục tăng — nếu không sẽ có hàng nghìn alert trùng.

Throttling Tarjan (engine.cpp:483). Tarjan có chi phí $O(V+E)$, chạy mỗi event sẽ giết throughput. Thay vào đó chỉ chạy mỗi scc_interval_seconds * 10 event:

```
if (total_events % (uint64_t)(cfg.scc_interval_seconds * 10) == 0) { ... }
```

Hệ số *10 là hằng số hardcoded chuyển "giây" → "event" (giả định ~10 event/giây) — comment thừa nhận đây là xấp xỉ proxy thay cho timer thật. Khi nontrivial > prev_nontrivial_sccs , engine **chỉ alert phần SCC mới** (so sánh số lượng, không so sánh thành phần cụ thể).

Một chi tiết tinh tế về thứ tự: BFS blast-radius (dòng 442-473) chạy trên đồ thị **trước khi** thêm edge của event hiện tại (edge được add_edge tại dòng 477-479, sau BFS) — nghĩa là blast radius không tính cạnh từ chính event đang xử lý.

4.2.2 Count-Min Sketch — ước lượng tần số xác suất

CountMinSketch (count_min_sketch.cpp) giữ bảng 2 chiều table_ kích thước depth_ × width_ . Khi đăng ký qua Registry (registry.cpp:18) tham số là width=2048, depth=5 → bộ nhớ $5 \times 2048 \times 8 = 81.920$ bytes ≈ 80 KB (memory_usage() dòng 84-86). Đây là đặc tính tiết kiệm bộ nhớ **$O(w \times d)$** so với HashMap exact $O(n)$ (n IP phân biệt không có chặn trên).

Constructor sinh depth seed tất định (count_min_sketch.cpp:12-16):

```
for (uint32_t i = 0; i < depth_; ++i)
    seeds_.push_back(0x9e3779b9 + i * 0x517cc1b7);
```

0x9e3779b9 là hằng số Golden Ratio, mỗi hàng dùng seed khác nhau → depth hàm băm **độc lập** (biến thể MurmurHash3). Sự độc lập này là điều kiện để bound lỗi hoạt động đúng.

record() tăng counter ở **mỗi** hàng (dòng 62-67); estimate() lấy **giá trị nhỏ nhất** trong depth hàng (dòng 69-76):

```
uint64_t min_val = std::numeric_limits<uint64_t>::max();
for (uint32_t i = 0; i < depth_; ++i) {
    uint32_t col = hash(key, key_len, seeds_[i]) % width_;
    min_val = std::min(min_val, table_[i][col]);
}
return min_val;
```

Tại sao dùng min : mỗi ô table_[i][col] có thể cộng dồn count của nhiều key bị **collision** tại cùng cột, nên table_[i][col] >= true_count(x) — chỉ **over-count**, không bao giờ under-count. Lấy min trên nhiều hàng giảm thiểu collision: xác suất tất cả depth hàng đều collision cao là rất nhỏ với các hàm băm độc lập. Đó là lý do gọi là "sketch" — ước lượng có độ chắc chắn cao (1-δ), không phải giá trị chính xác. Ngưỡng phát hiện port-scan là freq > 40 kết nối / source / cửa sổ 60s (engine.cpp:415 , đếm **volume** chứ không phải distinct ports — đây cũng là nguồn gốc hạn chế L-chatty ở §12). reset() (dòng 78-82) điền 0 toàn bảng (O(w×d), nhanh hơn recreate object).

Đính chính so với mô tả cũ: width/depth (2048/5) được hardcode trong factory lambda ở registry.cpp:18 , **không** đọc từ params.width / params.depth trong YAML — config có field này nhưng hiện chưa được truyền vào constructor.

4.2.3 Tarjan SCC — cài đặt iterative với explicit call stack

Thuật toán Tarjan gốc dựa trên DFS đệ quy. Với đồ thị mạng lớn (hàng nghìn IP), đệ quy sâu có thể **stack overflow**, nên tarjan.cpp chọn cách **iterative** với explicit call stack (comment dòng 15: *"Iterative DFS to avoid stack overflow on large graphs"*). Đây là đoạn code khó hiểu nhất của pipeline, cần đọc kỹ.

Ba cấu trúc dữ liệu: disc[u] (discovery time / DFN — gán một lần khi đỉnh lần đầu được thăm); low[u] (low-link — discovery time nhỏ nhất tiếp cận được từ cây con DFS gốc tại u , kể cả qua một back edge); on_stack[u] (đỉnh có đang trong Tarjan stack stk không — để phân biệt back edge với cross edge).

Mỗi "lời gọi đệ quy" được mô phỏng bằng một Frame (tarjan.cpp:17-21):

```
struct Frame {
    uint32_t u;
    std::vector<uint32_t> neighbors;
    size_t idx;    // chỉ số neighbor đang xử lý
};
```

Trường `idx` là **điểm then chốt**: khi "gọi đệ quy vào `v`", ta không thể dùng call stack của CPU (vì sẽ phải return về hàm), nên lưu `idx` để biết "lần sau quay lại frame này thì tiếp tục từ neighbor thứ `idx`".

Trong vòng lặp (`tarjan.cpp:42-88`), với mỗi frame trên đỉnh `call_stack` :

- Nếu **còn neighbor** (`idx < size`): lấy `v = neighbors[idx]` , tăng `idx++` , rồi phân nhánh:
 - `v` chưa thăm → **tree edge**: khởi tạo `disc[v]=low[v]=timer++` , đẩy `v` vào `stk` , tạo Frame mới cho `v` (mô phỏng "lệnh gọi đệ quy").
 - `v` đã thăm và `on_stack[v]` → **back edge**: `low[cur.u] = min(low[cur.u], disc[v])` (`tarjan.cpp:61-63`).
- Nếu **hết neighbor** (điểm "return" của lời gọi đệ quy, `tarjan.cpp:65-86`): kiểm tra `low[cur.u] == disc[cur.u]` . Điều kiện này nghĩa là `u` là **gốc của một SCC** (không back edge nào từ cây con của `u` vươn lên cao hơn `u`). Khi đúng, pop liên tục `stk` đến `u` (bao gồm) tạo thành một SCC:

```
if (low[cur.u] == disc[cur.u]) {
    Component comp; uint32_t w;
    do { w = stk.top(); stk.pop(); on_stack[w] = false; comp.push_back(w); }
    while (w != cur.u);
    sccs.push_back(std::move(comp));
}
```

Sau đó **truyền low-link ngược lên frame cha** thủ công (`tarjan.cpp:80-86`):

```
uint32_t u = cur.u; int u_low = low[u];
call_stack.pop();
if (!call_stack.empty())
    low[call_stack.top().u] = std::min(low[call_stack.top().u], u_low);
```

Đây là **điểm then chốt thứ hai**: trong đệ quy gốc, sau khi `strongconnect(v)` return, dòng `low[u] = min(low[u], low[v])` chạy tự động trong stack frame của `u` . Bản iterative phải làm thủ công — đọc `u_low` **trước** khi pop, rồi cập nhật frame cha **sau** khi pop. Engine phát cảnh báo `scc_anomaly` khi số SCC có `size() > 1` (nontrivial — biểu thị có cycle thực sự giữa các pod) tăng so với `prev_nontrivial_sccs` (`engine.cpp:493`); SCC kích thước 1 là trivial và bị bỏ qua.

4.2.4 BFS reachability — blast radius k-hop

BFSReachability::reachable_within() (bfs.cpp:15-27) là BFS chuẩn nhưng mỗi đỉnh mang kèm khoảng cách `d = result[u]`; khi `d >= max_hops` thì pop nhưng **không expand**:

```
while (!q.empty()) {
    uint32_t u = q.front(); q.pop();
    int d = result[u];
    if (d >= max_hops) continue;           // không vượt quá k hop
    for (uint32_t v : g.neighbors(u))
        if (result.find(v) == result.end()) { result[v] = d + 1; q.push(v); }
}
```

Kết quả là `unordered_map<uint32_t,int>` ánh xạ mỗi IP tiếp cận được sang số hop (`result[source]=0`). Ý nghĩa bảo mật: khi một IP bị phát hiện port-scan, engine tính ngay "nếu IP này bị chiếm, nó lateral-move tới tối đa bao nhiêu pod trong `max_hops` bước?".

Đỉnh chính so với mô tả cũ: `max_hops` **không** cố định là 3. Trong `registry.cpp`, BFS được đăng ký không kèm tham số `hardcode`; `max_hops` do engine truyền vào lúc gọi, lấy từ `cfg.reachability.max_hops` (`engine.cpp:375`, dùng tại dòng 443) — tức là cấu hình được trong YAML.

4.2.5 LPM Trie + IP classifier — longest prefix match

LPMtrie (`lpm_trie.cpp`) là **binary trie** theo từng bit IPv4 32-bit; mỗi Node có `children[0]` / `children[1]`, `label` (mặc định -1) và cờ `has_prefix`. Bit đi từ MSB xuống LSB qua `helper` (`lpm_trie.h:28-30`):

```
static int get_bit(uint32_t val, int pos) { return (val >> (31 - pos)) & 1; }
```

`insert()` (dòng 11-28) duyệt đúng `prefix_len` bit, đánh dấu node cuối `has_prefix=true` + gán `label`. `lookup()` (dòng 47-67) là longest-prefix-match điển hình: duyệt từ root theo bit của IP và **liên tục cập nhật** `best_label` mỗi khi gặp node `has_prefix`, dừng khi không còn nhánh con:

```
int32_t best_label = -1;
if (node->has_prefix) best_label = node->label; // root = /0 (default route)
for (int i = 0; i < 32; ++i) {
    int bit = get_bit(ip, i);
    if (!node->children[bit]) break;
    node = node->children[bit].get();
    if (node->has_prefix) best_label = node->label; // prefix dài hơn → ghi đè
}
return best_label;
```

Ví dụ: trie chứa 10.0.0.0/8 (label 1) và 10.244.0.0/16 (label 2), tra 10.244.1.5 cho best_label=1 tại depth 8 rồi =2 tại depth 16 → kết quả 2 (prefix cụ thể hơn thắng).

4.2.6 Registry — Strategy + Singleton template

Registry<Interface> (registry.h) là class template; mỗi instantiation tạo một singleton độc lập qua **Meyers' Singleton** (registry.h:18-21 , thread-safe theo C++11):

```
static Registry& instance() { static Registry r; return r; }
```

Bốn singleton tương ứng bốn interface (Estimator , SCCDetector , ReachabilityAnalyzer , IPClassifier) — hoàn toàn tách biệt, không chia sẻ map factories_ . register_algorithm() lưu một std::function<unique_ptr<Interface>()> (lambda factory). create(name) (registry.h:27-33) tra theo tên, ném runtime_error nếu không có, rồi gọi factory. Nhờ vậy để đổi Tarjan → Kosaraju chỉ cần sửa cfg.scc.name = "kosaraju" trong YAML, **không recompile**. Các lựa chọn swap được qua config:

Loại	Key config	Lựa chọn
Frequency estimator	frequency_estimator.name	count_min_sketch , hash_map_exact , heavy_hitters
SCC detector	scc_detector.name	tarjan , kosaraju
Reachability	reachability.name	bfs , dijkstra
IP classifier	ip_classifier.name	lpm_trie

Output của pipeline: alerts /var/log/zt/alerts.json (NDJSON), graph snapshot /var/log/zt/graph.json , stats /var/log/zt/stats.json .

4.3 K8sSecDaP-soc — Go / Python

Đường dẫn: K8sSecDaP/soc/ — gồm 4 service Go/Python + manifests.

```
soc/
├─ services/aggregator/          # Go: NATS consumer + correlator
├─ services/alert-bridge/        # Go: tail alerts.json → NATS
├─ services/incident-service/    # Go: REST API + NATS subscriber + K8s enforcer
├─ services/web-console/        # Python Flask: admin UI
├─ manifests/                   # K8s manifests (ArgoCD zt-soc-core)
└─ data/                        # CronJob manifests (ArgoCD zt-soc-data)
```

4.3.1 alert-bridge — tail file + phát hiện log rotation bằng inode

alert-bridge (alert-bridge/main.go) là sidecar cầu nối: pipeline C++ chỉ ghi NDJSON ra file (không nhúng thư viện NATS), sidecar này đọc file và publish lên NATS. Hàm cốt lõi readForever (dòng 73-109): mở file rồi Seek(0, io.SeekEnd) để nhảy tới cuối — tránh phát lại nội dung cũ mỗi lần restart, sau đó đọc từng dòng bằng bufio.ReadBytes('\n') .

Điểm thiết kế quan trọng nhất là xử lý io.EOF — khi chưa có dòng mới thì kiểm tra inode thay vì thoát:

```
if err == io.EOF {                                // alert-bridge/main.go:89-95
    time.Sleep(500 * time.Millisecond)
    if rotated(f, path) { return nil }
    continue
}

func rotated(f *os.File, path string) bool {      // dòng 128-135
    st1, _ := f.Stat()
    st2, _ := os.Stat(path)
    return !os.SameFile(st1, st2)
}
```

Tại sao cần kiểm tra inode: khi logrotate đổi tên alerts.json → alerts.json.1 rồi tạo file mới cùng tên, file descriptor đang mở vẫn trỏ inode cũ — đọc tiếp sẽ không thấy dữ liệu mới mà cũng không bao giờ báo lỗi. os.SameFile đối chiếu (dev, ino) của hai stat để phát hiện. Khi rotation xảy ra, readForever trả nil, vòng ngoài tailAndPublish sleep 2s rồi mở lại file từ đầu. Mỗi dòng hợp lệ được publish lên zt.alerts.raw; kết nối NATS dùng MaxReconnects(-1) (tự reconnect mãi mãi).

4.3.2 aggregator/correlator — dedup + nâng cấp severity cross-node

Correlator (aggregator/correlator.go) là thành phần phức tạp nhất của SOC. Trạng thái giữ trong map[string]*incidentState bảo vệ bởi sync.Mutex (dòng 16-22): khóa map là type + "|" + source (vd "port_scan|10.0.1.5"); trường Nodes là set map[string]struct{} các node K8s đã chứng kiến cùng nguồn tấn công trong cửa sổ.

Hàm Process (dòng 51-106) là điểm ra quyết định duy nhất, bọc hoàn toàn trong c.mu.Lock(). **Sliding window:** đồng hồ tính từ lần cuối thấy (LastSeenNS) chứ không phải lần đầu — incident sống chừng nào còn alert mới trong vòng window (mặc định 60s):

```

st, exists := c.incidents[key]
if !exists || now-st.LastSeenNS > c.Window.Nanoseconds() {
    st = &incidentState{...}; c.incidents[key] = st
    out = a; out.Correlation = c.snapshot(st, false)
    return out, true // incident mới HOẶC window hết hạn → phát
}

```

Cross-node upgrade — đoạn tính tế nhất (dòng 86-105):

```

prevCrossNode := len(st.Nodes) >= 2
st.LastSeenNS = now; st.Count++
if a.NodeName != "" { st.Nodes[a.NodeName] = struct{}{} }
nowCrossNode := len(st.Nodes) >= 2

if !prevCrossNode && nowCrossNode { // vừa chuyển sang đa-node
    out = a
    out.Severity = upgradeSeverity(a.Severity)
    out.Tags = appendUnique(out.Tags, "cross_node_attacker")
    out.Correlation = c.snapshot(st, true)
    return out, true
}
return nil, false // trùng lặp thuần → DROP

```

Hành vi rất có chủ ý: chỉ **alert đầu tiên** (lúc tạo entry) và alert đánh dấu *khoảnh khắc* cùng source xuất hiện trên **node thứ hai** mới được phát xuống. Nếu 100 alert cùng 10.0.1.5 trên cùng một node trong 60s, chỉ 1 đi tiếp; nếu trong đó có một alert từ node thứ hai, có thêm đúng 1 alert nữa với severity nâng theo thang low→medium→high→critical (upgradeSeverity , dòng 162-171) và tag cross_node_attacker . Goroutine cleanupLoop (dòng 125-136) đánh thức mỗi window gọi evictExpired xóa entry quá hạn. An toàn goroutine: Process , evictExpired , Stats đều chiếm c.mu ; NowNS được làm injectable (field thay vì gọi time.Now() trực tiếp) để test được.

4.3.3 incident-service — thứ tự khởi động và APPLY_MODE

main() (incident-service/cmd/server/main.go) khởi động có chủ đích:

1. **Postgres + retry** (connectWithRetry dòng 208-224): thử tối đa 30 lần × 2s, rồi store.New chạy applyMigrations tuần tự các file SQL nhúng trong binary.
2. **Applier theo APPLY_MODE** (mặc định dryrun): dryrun → DryRunApplier chỉ ghi YAML ra /var/lib/zt/netpol-drafts ; apply → K8sApiApplier thực sự tạo/cập nhật NetworkPolicy qua in-cluster ServiceAccount. Đây là công tắc an toàn — staging luôn dryrun .
3. **PodOps có điều kiện** (dòng 81-89): chỉ khi APPLY_MODE=apply mới gọi response.NewPodOps() ; nếu dryrun thì podOps == nil và route cần nó trả 400 → ngăn thao tác pod ngoài production.

4. **SMTP notifier** (no-op nếu `SMTP_HOST` rỗng), **MinIO sink** (chỉ nếu `MINIO_ENDPOINT` có giá trị, dòng 102-112).
5. **HTTP API** :8080 + **metrics** :9101 (mỗi cái một goroutine), cuối cùng **natssub.Run** trong goroutine (dòng 156-163) — đặt sau HTTP server để `/health` sẵn sàng trước khi alert tới.

4.3.4 natssub/subscriber.go — xử lý alert enriched + auto-gen action

Run (`natssub/subscriber.go:56`) subscribe `zt.alerts.enriched` theo queue group `incident-service` . Mỗi message chạy chuỗi xử lý tuần tự:

1. **UpsertIncident** (dòng 94): SQL `INSERT ... ON CONFLICT` (chi tiết §4.3.7), trả `Incident` + cờ `created bool` .
2. **AppendAlert** (dòng 100): luôn lưu JSON gốc của alert vào bảng `alerts` (bản ghi forensic).
3. **Audit + notify khi created** (dòng 103-114): `aud.Record(actor="system")` + `notif.NotifyIncident` email cho SOC.
4. **Làm giàu pod context** (dòng 120-129): chỉ khi `created` và `resolver != nil` (PodOps ở apply mode), `LookupPodByIP("zt-targets", inc.SourceIP)` → `SetIncidentPodContext` cập nhật `pod_name` / `pod_namespace` để analyst biết ngay pod tấn công.
5. **Auto-generate quarantine action** (dòng 132-165):

```
if _, ok := autoSet[inc.Severity]; ok {           // high hoặc critical
    actions, err := st.ListActions(ctx, "pending", 200)
    pendingForThisIncident := false
    for _, a := range actions {
        if a.IncidentID == inc.ID { pendingForThisIncident = true; break }
    }
    // chỉ tạo action mới nếu CHƯA có pending action cho incident này
}
```

Điều kiện kép — `severity ∈ {high, critical}` **và** chưa có action `pending` — tránh việc mỗi alert lặp (vd khi correlator emit lần hai do cross-node upgrade) lại sinh thêm một action trùng. Nội dung action là YAML `NetworkPolicy` do `response.GenerateQuarantine` sinh, lưu vào cột `yaml_draft` .

4.3.5 response/networkpolicy.go — quyết định kiến trúc egress-deny

`GenerateQuarantine` (`response/networkpolicy.go:45-79`) sinh `NetworkPolicy` dạng:

```
spec:
  podSelector:
    matchLabels:
      zt-soc-quarantine-<id>: "true"  # chọn attacker pod (label gán lúc enforce)
  policyTypes: [Egress]
  egress: []                          # chặn TẤT CẢ egress
```

Tại sao egress-deny trên attacker, không phải ingress-allow trên victim — package comment giải thích trực tiếp (dòng 9-15):

```
// Kubernetes NetworkPolicies are ADDITIVE – a connection is permitted if ANY
// policy selecting the pod allows it. An "ingress allow 0.0.0.0/0 except
// attacker" is therefore silently defeated by any broader allow policy on
// the same target pods. Restricting the attacker Pod's own egress cannot be
// overridden by ingress policies and stops port-scan / lateral movement at source.
```

Điểm mấu chốt là tính **additive** của NetworkPolicy: một connection được phép nếu **bất kỳ** policy nào chọn pod đó cho phép. Nếu victim đã có sẵn một policy ingress allow from all (để phục vụ web traffic), thì policy mới ingress deny from attacker sẽ vô hiệu vì policy cũ vẫn allow. Ngược lại, egress: [] (deny-all egress) trên chính attacker pod **không thể bị ingress policy nào của victim hủy** — vì ingress và egress là hai chiều độc lập. Attacker mất khả năng kết nối ra ngoài ngay tại node. podSelector không dùng IP (K8s không hỗ trợ chọn pod theo IP) mà dùng label zt-soc-quarantine-<id>=true, gán vào pod ở bước thực thi action (§4.3.6).

4.3.6 response/podops.go — thao tác pod qua client-go

PodOps (response/podops.go) bọc các thao tác K8s cấp pod qua interface PodClient :

Method	Hành động (file:line)
LookupPodByIP(ns, ip)	List pod trong ns, so pod.Status.PodIP == ip ; trả (name, found, err) — found=false khác với lỗi (dòng 84-95)
QuarantinePodByIP(ns, ip, key)	Lookup rồi StrategicMergePatch chỉ merge metadata.labels[key]="true", không xóa label cũ → pod vẫn hoạt động nhưng đã có label cho NetworkPolicy (dòng 102-123)
DeletePod(ns, name)	Get kiểm tra tồn tại rồi Delete với gracePeriodSeconds=0 (xóa ngay) (dòng 38-49)
ShutdownPod(ns, name)	Patch label zt-soc/shutdown=true rồi Delete với grace 300s — cho analyst kịp kubectl exec thu logs (dòng 54-79)

NewPodOps (dòng 134-140) dùng rest.InClusterConfig() lấy ServiceAccount token mount sẵn trong pod (/var/run/secrets/kubernetes.io/serviceaccount/token) — cơ chế chuẩn của client-go cho ứng dụng chạy trong cluster.

4.3.7 store — schema và câu UpsertIncident (trick xmax)

Bốn bảng (migrations/001_init.sql): incidents (mỗi hàng một incident, UNIQUE(incident_key) là cơ sở dedup), alerts (raw JSON, incident_id REFERENCES incidents(id) ON DELETE CASCADE), actions

(vòng đời pending→approved/rejected→executed/failed , tự hết hạn

expires_at DEFAULT NOW()+INTERVAL '24 hours'), audit_log (bất biến, chỉ INSERT). Migration 004_incident_pod_context.sql thêm hai cột nullable pod_name / pod_namespace (dùng ADD COLUMN IF NOT EXISTS cho idempotent).

UpsertIncident (store/postgres.go:114-138) là phần phức tạp nhất, dùng

INSERT ... ON CONFLICT (incident_key) DO UPDATE :

```
INSERT INTO incidents (incident_key, type, source_ip, severity, ...) VALUES ($1,$2,$3,$4,'new')
ON CONFLICT (incident_key) DO UPDATE SET
    severity = CASE WHEN severityRank($4) > severityRank(incidents.severity)
                    THEN $4 ELSE incidents.severity END,
    nodes_observed = (SELECT array_agg(DISTINCT n) FROM unnest(incidents.nodes_observed || $5) AS n),
    last_seen_ns = GREATEST(incidents.last_seen_ns, $7),
    occurrence_count = incidents.occurrence_count + 1,
    cross_node = incidents.cross_node OR $11,
    tags = (SELECT array_agg(DISTINCT t) FROM unnest(incidents.tags || $10) AS t),
    updated_at = NOW()
RETURNING ..., (xmax = 0) AS inserted
```

Ba điểm đáng chú ý: (1) **Severity chỉ được nâng, không bao giờ hạ** —

CASE WHEN severityRank(\$4) > severityRank(incidents.severity) với severityRank định nghĩa ở migration 001 (critical=4 > high=3 > medium=2 > low=1). (2) **Gộp nodes/tags không trùng** bằng unnest(... || ...) + array_agg(DISTINCT ...). (3) **Phân biệt INSERT vs UPDATE** qua

(xmax = 0) AS inserted : khai thác MVCC của PostgreSQL — hàng vừa INSERT có xmax=0 , hàng vừa UPDATE có xmax là transaction ID. Đây là cách duy nhất biết trong một ON CONFLICT DO UPDATE liệu hàng có mới được tạo hay không mà không cần truy vấn thêm — chính cờ created mà subscriber dùng để quyết định audit/notify.

4.3.8 api.go — RBAC Bearer token và luồng approve

RBAC (rbac/rbac.go) dùng token tĩnh parse từ env RBAC_TOKENS định dạng user:role:token,... ; middleware đọc Authorization: Bearer <token> , tra map, inject user / role vào context. Thứ bậc role: viewer(1) < editor(2) < admin(3) . Các route nhảy cảm yêu cầu admin (api.go:53-55):

```
r.With(rbac.RequireRole(rbac.RoleAdmin)).Post("/{id}/approve", s.approveAction)
r.With(rbac.RequireRole(rbac.RoleAdmin)).Post("/{id}/reject", s.rejectAction)
```

Luồng approveAction (dòng 241-277): (1) DecideAction(id, "approve", user) UPDATE

status='approved' WHERE id=\$1 AND status='pending' — điều kiện pending ngăn approve lại action đã xử lý. (2) executeAction dispatch theo a.Type : quarantine_pod → QuarantinePodByIP gán label (nếu PodOps có) rồi Applier.Apply tạo NetworkPolicy (nếu pod chưa tìm thấy, Apply vẫn tiếp tục — policy

tồn tại sẵn và có hiệu lực ngay khi pod tái tạo với label đó); `delete_pod / shutdown_pod` → decode `yaml_draft` thành `PodOpPayload` rồi gọi tương ứng. (3) `MarkActionExecuted` ghi `executed / failed`. (4) `aud.Record audit`. Một điểm tế nhị: lỗi từ `executeAction` **không** abort — action vẫn được mark và response 200 kèm trường `error` trong JSON, để analyst thấy policy đã ghi nhận dù pod không tìm thấy để label.

4.3.9 web-console/app.py — thin proxy, bảo mật token

Web console (`web-console/app.py`) là Flask proxy giữa browser admin và REST API của incident-service.

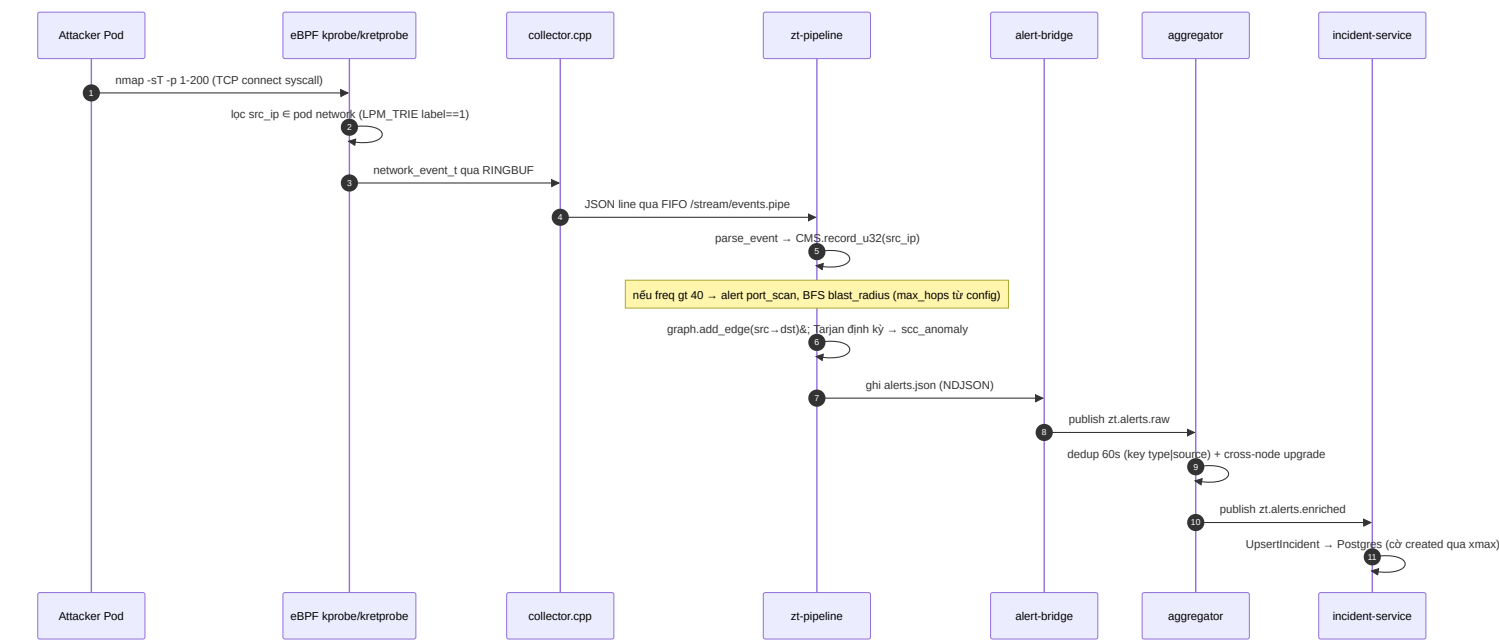
Xác thực hai lớp: browser dùng HTTP Basic Auth (`ADMIN_USERNAME / ADMIN_PASSWORD`) — decorator `_require_basic_auth` (dòng 39-50) chặn mọi route. Sau khi qua Basic Auth, mọi call tới incident-service được thực hiện **server-side** với Bearer token đọc từ `RBAC_ADMIN_TOKEN` :

```
def _api(method, path, **kwargs):                                # app.py:54-59
    headers = kwargs.pop("headers", {})
    headers["Authorization"] = f"Bearer {ADMIN_TOKEN}"
    return requests.request(method, INCIDENT_API.rstrip("/") + path, headers=headers, ...)
```

Lợi ích: `RBAC_ADMIN_TOKEN` **không bao giờ lộ ra browser** — chỉ là env của Flask pod, dùng trong call server-to-server; browser chỉ biết username/password Basic Auth. Khi approve/reject, Flask không reload toàn trang mà render partial `_action_row.html` (dòng 176-178), HTMX swap fragment vào DOM. Route `/audit/export/proxy` (dòng 148-158) dùng `stream=True` để stream CSV theo chunk 8KB, tránh OOM khi export audit log lớn.

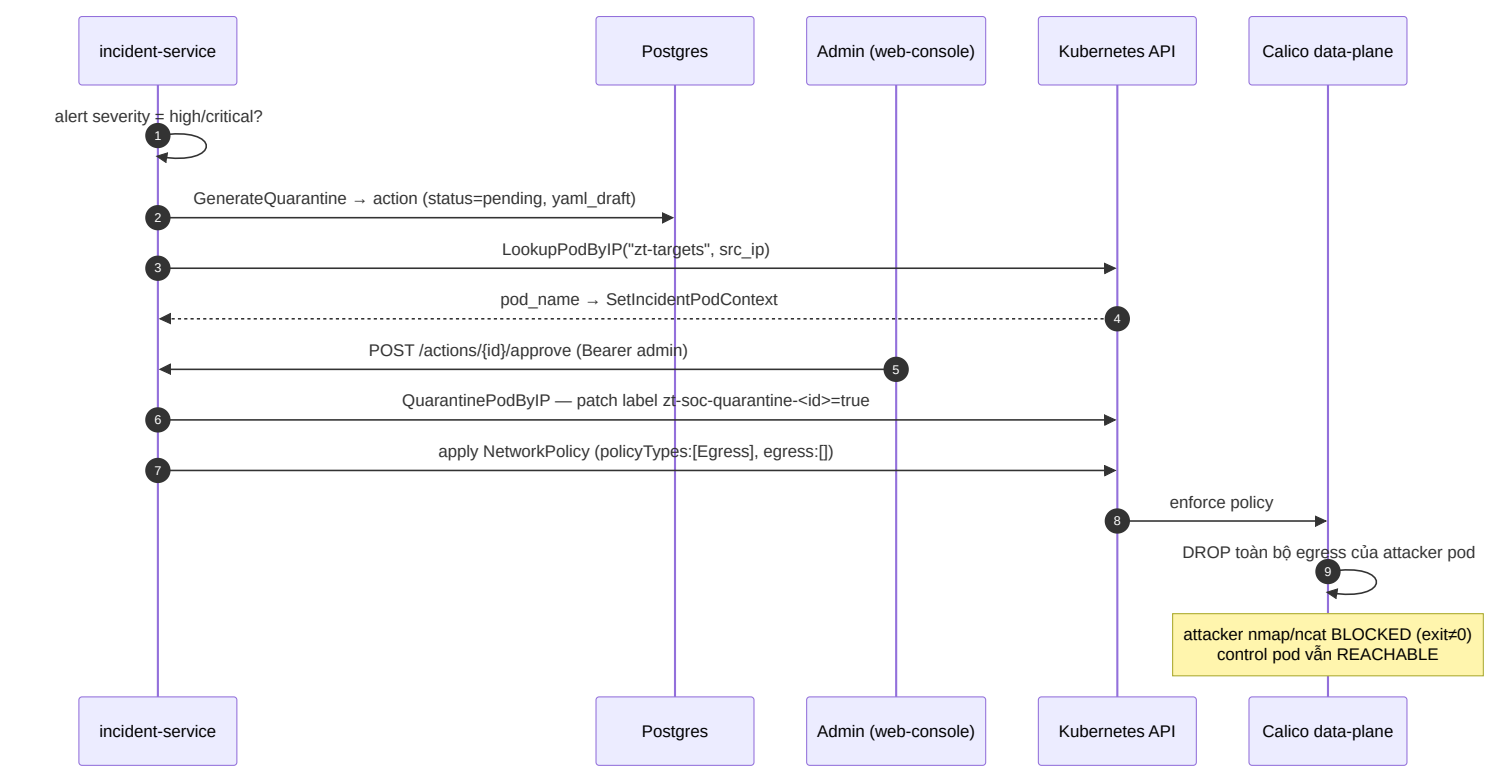
5. Luồng Thực Thi End-to-End

5.1 Luồng Phát Hiện (Detect)



Nguồn đường eBPF: `soc/manifests/23-pipeline-ebpf.yaml` .

5.2 Luồng Ngăn Chặn (Prevent)



Kết quả đo thực tế (2026-06-06): trước quarantine attacker → web:80 reachable (exit 0); sau quarantine BLOCKED; control pod không bị ảnh hưởng. Nguồn: `plans/plan-thesis.md:182-183` .

6. Config Platform

6.1 Infra — Ansible

Đường dẫn: K8sSecDaP/infra/ansible/ . Cluster gồm 3 node vật lý nối qua **Tailscale** (overlay VPN):

Node	Role	Tailscale IP
userver-master	Control Plane	100.120.16.113
userver-worker-home	Worker	100.119.246.28
userver-worker-1	Worker	100.95.126.102

Nguồn: infra/ansible/inventory/hosts.yml . Lưu ý: userver-worker-home kết nối qua DERP relay → latency cao hơn, ảnh hưởng Prometheus scrape (deploy/platform/argocd/apps/monitoring.yaml:44-54 , scrapeTimeout=45s).

Playbook site.yml chia 4 phase: (1) tất cả node — tailscale , common (kernel modules, sysctl), containerd , kubernetes , cni_plugins , firewall ; (2) master — master_init (kubeadm init), cni_calico ; (3) worker — worker_join ; (4) master — verify kubectl get nodes/pods .

Role cni_calico (infra/ansible/roles/cni_calico/tasks/main.yml:74-87): copy + chạy install-calico.sh (idempotent), chờ DaemonSet calico-node rollout (300s) + CoreDNS ready, **verify MTU = 1230** (Tailscale overlay yêu cầu reduced MTU — fail nếu khác), verify node InternalIP = Tailscale IP. Makefile: make setup/master/worker/reset/ping/vault-edit .

6.2 Deploy / GitOps — ArgoCD

Bootstrap một lần bằng helmfile (deploy/platform/helmfile/helmfile-bootstrap.yaml), theo thứ tự phụ thuộc:

1. cert-manager (v1.16.3)

← TLS foundation
2. traefik (v34.3.0)

← Ingress, cần cert-manager
3. sealed-secrets (v2.17.1)

← Encrypted secrets in Git
4. argocd (v7.8.7)

← GitOps controller, cần traefik + sealed-secrets

Sau bootstrap: kubectl apply -f platform/argocd/app-of-apps.yaml (root app, source platform/argocd/apps/ , prune:true selfHeal:true). ArgoCD tự tạo một Application cho mỗi file YAML trong thư mục đó.

Sync waves (apps/zt-soc.yaml:28-29): wave -10 zt-cni-calico (CNI phải sẵn trước, **prune:false** — không bao giờ để ArgoCD tự xóa CNI), wave 0 zt-soc-core (NATS, Postgres, pipeline DaemonSet, services), wave +10 zt-soc-data (CronJobs backup/retention/weekly-report). ApplicationSet

appsets/cluster-roots.yaml generate app-of-apps cho mỗi cluster từ clusters/*/config.yaml (template mở rộng GKE tương lai).

Các Application khác: monitoring (kube-prometheus-stack 68.4.5), kyverno (3.3.7) + kyverno-policies, traefik, cloudflared, dns, grafana-dashboards, ingress-routing, logging (Loki), metrics-server, tekton (đang xem xét disable), uptime-kuma; harbor / rancher đang disabled.

6.3 Security — Chính Sách & RBAC

Kyverno ClusterPolicies (deploy/platform/argocd/policies/baseline.yaml , đều mode audit): require-managed-by-label (mọi workload phải có label app.kubernetes.io/managed-by), disallow-latest-tag (Pod không dùng tag :latest).

RBAC incident-service (soc/manifests/05-rbac-incident-service.yaml): ServiceAccount zt-incident-service (ns soc) bind vào Role zt-incident-responder **chỉ trong ns zt-targets** với quyền NetworkPolicies (full CRUD), Pods (get/list/watch/delete/patch), Events (get/list) — **least-privilege**, không có quyền cluster-wide. RBAC tăng ứng dụng: env RBAC_TOKENS (user:role:token, ...), web-console dùng RBAC_ADMIN_TOKEN proxy (cmd/server/main.go:35).

7. Cơ Sở Dữ Liệu và Lưu Trữ

7.1 PostgreSQL Schema

Chi tiết câu UpsertIncident và trick xmax xem §4.3.7. Tóm tắt bốn bảng (soc/services/incident-service/internal/store/migrations/001_init.sql):

incidents — incident_key (UNIQUE, type|source_ip), type (port_scan / blast_radius / scc_anomaly), source_ip, severity (low/medium/high/critical), status (new/ack/resolved/suppressed), nodes_observed (TEXT[]), blast_radius (JSONB), cross_node (BOOL), pod_name / pod_namespace (migration 004).

alerts — raw JSON từng alert, liên kết incident_id (ON DELETE CASCADE).

actions — type (quarantine_pod / delete_pod / shutdown_pod), status (pending/approved/rejected/executed/expired/failed), yaml_draft (TEXT), expires_at (mặc định 24h).

audit_log — bất biến, mọi hành động admin/system.

7.2 MinIO Object Storage

Buckets: `zt-postgres-backups` (backup DB), `zt-reports` (báo cáo tuần HTML), `zt-pcap-dumps` (packet captures nếu có), `zt-netpol-archives` (NetworkPolicy đã apply). Nguồn: `soc/scripts/demo-e2e.sh:182` .

7.3 CronJobs Data Layer

`soc/data/` (ArgoCD `zt-soc-data`): `20-postgres-backup-cronjob.yaml` , `25-weekly-report-cronjob.yaml` , `30-retention-cronjob.yaml` .

8. Observability

8.1 Prometheus Metrics

Aggregator (`:9100/metrics` , `aggregator/main.go:42-66`): `zt_aggregator_alerts_received_total` , `..._emitted_total` (label reason), `..._suppressed_total` , `zt_aggregator_active_incidents` (Gauge), `..._active_cross_node_incidents` .

Incident-service (`:9101/metrics` , `natssub/subscriber.go:21-36`): `zt_incident_alerts_received_total` , `zt_incidents_created_total` , `..._updated_total` , `zt_actions_generated_total` , `zt_incident_subscriber_errors_total` .

8.2 Alert Rules (`monitoring.yaml:88-107`)

Alert	Điều kiện	Severity
ZTIncidentRateHigh	<code>rate(zt_incidents_created_total[5m]) > 0.1</code>	warning
ZTActionsPendingBacklog	pending actions > 5 trong 5 phút	critical

8.3 Grafana & Cloudflare Tunnel

Dashboard (`soc/manifests/grafana/`): `zt-soc-overview.json` , `zt-soc-incidents.json` . Service expose qua Cloudflare Tunnel (`deploy/platform/cloudflared/ingress/`): ArgoCD, Grafana, Prometheus, Uptime Kuma, web-console.

9. Dependencies và Công Cụ

9.1 Phiên Bản Đã Pin

Thành phần	Phiên bản	Nguồn
Kubernetes	1.32	versions.yml:10
Calico CNI	v3.28.2	versions.yml:21
cricctl	v1.32.0	versions.yml:17
CNI plugins	v1.6.2	versions.yml:19
ArgoCD chart	7.8.7	helmfile-bootstrap.yaml:67
cert-manager	1.16.3	helmfile-bootstrap.yaml:44
Traefik	34.3.0	helmfile-bootstrap.yaml:53
Sealed Secrets	2.17.1	helmfile-bootstrap.yaml:59
kube-prometheus-stack	68.4.5	monitoring.yaml:15
Kyverno	3.3.7	kyverno.yaml:19

9.2 Stack Công Nghệ

Kernel/eBPF (Linux ≥5.8 BTF, libbpf, BPF CO-RE, clang/llvm) · Collector (C++17 + libbpf-dev) · Pipeline (C++17, CMake, GTest, Google Benchmark) · SOC (Go: aggregator/alert-bridge/incident-service; Python Flask: web-console) · NATS · PostgreSQL (pgx) · MinIO (minio-go) · containerd · Calico (Tigera Operator) · Tailscale · Kubernetes 1.32 (kubeadm) · ArgoCD (app-of-apps) · Ansible · Helmfile/Helm · Traefik · cert-manager · Sealed Secrets · Kyverno · GitHub Actions (collector build) + Tekton · Prometheus/Grafana/Alertmanager/Loki/Uptime Kuma · MailHog · Cloudflare Tunnel · LaTeX (XeLaTeX + Biber).

10. Hướng Dẫn Build và Chạy

10.1 Trình Tự Đọc Mã Nguồn Gợi Ý

- plans/plan-thesis.md — toàn bộ kiến trúc và trạng thái hệ thống.
- Sơ đồ Mục 3 + luồng Mục 5 của tài liệu này.

3. eBPF: collector/src/kern/tcp_connect.bpf.c (ngắn, ~148 dòng) → collector/src/user/collector.cpp → §4.1.
4. Pipeline: pipeline/pipeline/src/engine.cpp từ main() dòng 221 → §4.2 (chú ý kỹ §4.2.3 Tarjan iterative).
5. SOC: incident-service/cmd/server/main.go → internal/natssub/subscriber.go → internal/response/networkpolicy.go → §4.3.
6. Deploy: deploy/platform/argocd/app-of-apps.yaml → apps/zt-soc.yaml .
7. Demo: soc/scripts/demo-e2e.sh (8 bước chứng minh toàn luồng).

10.2 Yêu Cầu Tối Thiểu

Linux kernel ≥ 5.8 với BTF (/sys/kernel/btf/vmlinux tồn tại) · Docker/podman · clang , llvm , libbpf-dev , bpftool (build collector) · CMake ≥ 3.20 + GCC/Clang C++17 (build pipeline) · Go ≥ 1.21 (SOC) · Python ≥ 3.10 (web-console).

10.3 Build Collector (eBPF)

```
cd K8sSecDaP/collector
./build.sh          # 1) copy /sys/kernel/btf/vmlinux → btf/vmlinux  2) docker build -t collect
# hoặc dùng image CI:
docker pull johnnaeder/ebpf-portscan-k8s-check:collector-ebpf-s3
```

10.4 Build Pipeline (C++)

```
cd K8sSecDaP/pipeline
mkdir -p libdsa/build && cd libdsa/build
cmake .. -DCMAKE_BUILD_TYPE=Release && make -j$(nproc)
ctest --output-on-failure          # chạy test
cd K8sSecDaP/pipeline && mkdir -p build && cd build
cmake .. -DCMAKE_BUILD_TYPE=Release && make zt-pipeline
```

Chạy với dữ liệu test:

```
python K8sSecDaP/scripts/generate_synthetic_events.py > /tmp/events.json
# sửa config/pipeline.yaml: event_source.type="file", path="/tmp/events.json"
./build/zt-pipeline --config K8sSecDaP/config/pipeline.yaml  # output: output/alerts.json
```

10.5 Build SOC Services

```
cd K8sSecDaP/soc/services/aggregator      && go build ./... && go test ./...
cd K8sSecDaP/soc/services/incident-service && go build ./cmd/server/... && go test ./...
cd K8sSecDaP/soc/services/web-console     && pip install -r requirements.txt && python ap
```

10.6 Setup Cluster + Deploy Platform

```
cd K8sSecDaP/infra/ansible
make sync && make check && make setup      # (make reset && make setup nếu cần cài lại)

cd K8sSecDaP/deploy/platform/helmfile
helmfile -f helmfile-bootstrap.yaml apply  # bootstrap một lần
kubectl apply -f ../argocd/app-of-apps.yaml # ArgoCD tự sync tất cả
```

10.7 Demo + Lệnh Vận Hành Thường Dùng

```
cd K8sSecDaP/soc/scripts && chmod +x demo-e2e.sh && ./demo-e2e.sh  # 8 bước; ./demo-e2e.sh :

kubectl -n zt-mapper get pods -o wide
kubectl -n zt-mapper logs <pod> -c collector -f                # eBPF events
kubectl -n zt-mapper logs <pod> -c pipeline | grep ALERT        # alert output
kubectl -n soc logs deploy/zt-incident-service | grep incident.created
kubectl -n zt-targets get networkpolicy | grep quarantine
kubectl -n soc port-forward svc/zt-web-console 5000:5000      # http://localhost:5000 (admin/admi

# Kiểm tra eBPF trên node:
sudo bpftool prog show | grep -A4 -i 'kprobe\|tcp_v4_connect'
sudo bpftool map show  | grep -i 'cidr_map\|events\|sock_store'
sudo bpftool map dump name cidr_map | head
```

11. Kết Quả Đánh Giá Đã Đo Được

Số liệu đo trên cluster thật (userver-worker-1, kernel 6.8).

11.1 Detection & False Positive Rate

Kịch bản	Loại	Kết quả
B1-B5: benign 15–38 conn/60s	benign	Không alert (FPR=0/6)
B-edge: boundary 38 conn	benign	Không alert
S1: vertical scan (>40 conn/60s)	attack	DETECTED
S2: lateral scan (multi-target)	attack	DETECTED (blast_radius)
S3: burst scan	attack	DETECTED
S4: boundary-over (45 conn)	attack	DETECTED
L-slow: slow scan (<40/window)	limit-FN	MISSED (biết trước)
L-chatty: benign >40 conn/window	limit-FP	FALSE POSITIVE (biết trước)

Detection rate: 4/4 (100%) · FPR: 0/6 (0%) · Nguồn:
soc/scripts/eval/results/eval-20260608-010448.md

11.2 Overhead eBPF

- **Per-connect:** 1544 ± 126 ns/connect (N=10, 95% CI) — kprobe entry 168 ± 22 ns, kretprobe return 1376 ± 106 ns.
- **Tổng probe overhead tại 1316 connect/s:** ~0.225% một CPU core.
- **Userspace collector:** ~1m CPU, ~3Mi memory (`kubectl top`).

Nguồn: soc/scripts/eval/results/overhead-stats-20260608-014723.md

11.3 Latency Detect → Prevent

SOC machine-response: avg 91 ms · p50 54 ms · p95 234 ms (thời gian máy alert → NetworkPolicy draft, chưa gồm human review). Nguồn: `soc/scripts/latency-report.sh` , `latency-report.sql` .

12. Các Vấn Đề Đã Biết và Hạn Chế

12.1 Hạn Chế Kỹ Thuật

Vấn đề	Mô tả	Hướng xử lý
L-slow (FN)	Slow scan <40 conn/window né được tumbling window	Sliding-window CMS (STRETCH)
L-chatty (FP)	Benign client >40 conn/60s bị gấn cò (CMS đếm volume , không đếm distinct ports — xem §4.2.2)	Key CMS theo distinct dst_port
kprobe chỉ bắt connect ()	nmap -sS (raw SYN, cần root) không qua tcp_v4_connect ; nmap -sT thì bắt được	Thêm XDP/tc hook bắt raw SYN (STRETCH)
Quarantine pod tái tạo	Attacker do Deployment quản lý → pod bị xóa tái tạo IP mới → cần re-quarantine (xem DeletePod §4.3.6)	Ghi nhận như hạn chế thiết kế
IPv4 only	Collector chỉ hook tcp_v4_connect	Ngoài scope đề tài

Nguồn: plans/plan-thesis.md:251-252 , collector comment dòng 8.

12.2 Vấn Đề Vận Hành Đã Biết

Vấn đề	Mô tả
Grafana datasource no-data	Một số panel thiếu dữ liệu — cần kiểm tra datasource config
kube-proxy/etcd/scheduler metrics down	Endpoint trả 0 do firewall/bind address
DERP relay latency worker-home	userver-worker-home qua Tailscale DERP → scrape timeout cao hơn
Hardcoded path trong engine.cpp	Dòng 246: /home/john/dev/Gradution-Reports/ trong list config candidates — cần xóa trước release

Nguồn: plans/plan-thesis.md:190-191 , engine.cpp:246.

12.3 Phần Chưa Hoàn Thiện

Báo cáo BC2 (stage2 LaTeX) đang viết · Eval định lượng bổ sung (overhead-vs-load curve, latency histogram) chưa có · So sánh công cụ (Falco, Tetragon, Cilium, KubeArmor) mới sơ bộ · Disable Tekton chưa thực hiện · Ảnh chụp app thật trong báo cáo đang TODO.

13. Glossary — Thuật Ngữ Viết Tắt

Thuật ngữ	Giải nghĩa
eBPF	extended Berkeley Packet Filter — framework chạy code an toàn trong kernel Linux
CO-RE	Compile Once – Run Everywhere — kỹ thuật eBPF portable across kernel versions (xem §4.1.2)
BTF	BPF Type Format — thông tin kiểu dữ liệu kernel, nền tảng cho CO-RE
kprobe / kretprobe	Kernel probe — hook vào entry/return của kernel function
LPM Trie	Longest Prefix Match Trie — tra IP subnet hiệu quả (binary trie theo bit, xem §4.2.5)
CMS	Count-Min Sketch — cấu trúc xác suất ước lượng tần suất, bộ nhớ $O(w \times d)$ (xem §4.2.2)
SCC	Strongly Connected Component — thành phần liên thông mạnh trong đồ thị có hướng
BFS	Breadth-First Search — duyệt đồ thị theo chiều rộng (k-hop, xem §4.2.4)
Tarjan	Thuật toán tìm SCC $O(V+E)$ bằng DFS với low-link; ở đây cài iterative với explicit call stack (§4.2.3)
Kosaraju	Thuật toán tìm SCC khác (hai DFS); thay thế Tarjan qua Registry
CNI / Calico	Container Network Interface; Calico hỗ trợ NetworkPolicy enforcement (iptables/eBPF data-plane)
NATS	Message broker hiệu năng cao, pub/sub
GitOps / ArgoCD / App-of-Apps	Git là single source of truth; ArgoCD controller; một Application quản lý nhiều Application
Helmfile	Tool declarative quản lý nhiều Helm release
Tailscale / DERP	VPN mesh (WireGuard) nối node; DERP = relay server khi P2P không khả dụng
Kyverno	Admission controller K8s — enforce policy bằng YAML
MinIO	Object storage tương thích S3, self-hosted

Thuật ngữ	Giải nghĩa
Sealed Secrets	Bitnami tool mã hóa Secret để lưu an toàn vào Git
DaemonSet	K8s resource chạy một Pod trên mỗi node
NetworkPolicy	K8s resource định nghĩa luật network cho Pod (additive — xem §4.3.5)
RBAC	Role-Based Access Control
SOC	Security Operations Center
FPR / FN	False Positive Rate / False Negative
ĐATN / BC1 / BC2	Đồ Án Tốt Nghiệp / Báo cáo 1 / Báo cáo 2 (thesis chính)
DSA	Data Structures and Algorithms
ZT / Zero-Trust	"Không tin tưởng mặc định", xác thực mọi kết nối
RING BUFFER	Vùng nhớ vòng, stream event kernel → user hiệu quả (xem §4.1.5)
FIFO	Named pipe — truyền data giữa container collector và pipeline
CO-RE relocation	Quá trình libbpf patch offset lúc load theo BTF kernel target
Blast Radius	Tập pod có thể bị ảnh hưởng nếu một pod bị chiếm (BFS k-hop)
Lateral Movement	Dịch chuyển ngang giữa các pod trong cùng cluster
human-in-the-loop	Yêu cầu con người phê duyệt trước khi hệ thống thực thi hành động tự động
MVCC / xmax	Multi-Version Concurrency Control; xmax=0 để phân biệt INSERT vs UPDATE (xem §4.3.7)